

Overloading

Overloading

Come definire piu metodi con lo stesso nome ma parametri diversi.

Lo stesso nome, parametri diversi

In Java puoi avere piu metodi con lo stesso nome, se hanno parametri diversi.

Questa tecnica si chiama **overloading**.

```
public static void saluta() {  
    System.out.println("Ciao!");  
}  
  
public static void saluta(String nome) {  
    System.out.println("Ciao, " + nome + "!");  
}
```

Java sceglie quale metodo chiamare guardando gli argomenti.

Un primo esempio completo

```
public class Saluti {  
    public static void saluta() {  
        System.out.println("Ciao!");  
    }  
  
    public static void saluta(String nome) {  
        System.out.println("Ciao, " + nome + "!");  
    }  
  
    public static void main(String[] args) {  
        saluta();  
        saluta("Luca");  
    }  
}
```

Output:

```
Ciao!  
Ciao, Luca!
```

La prima chiamata non passa argomenti. La seconda passa una `String`.

La firma del metodo

La **firma** di un metodo è composta da:

- nome del metodo
- tipi dei parametri
- ordine dei parametri

Questi metodi hanno firme diverse:

```
public static void stampa(String testo) {  
    System.out.println(testo);  
}  
  
public static void stampa(int numero) {  
    System.out.println(numero);  
}
```

Uno riceve una `String`, l'altro un `int`.

Il valore di ritorno non basta

Non puoi distinguere due metodi solo dal tipo di ritorno.

Questo codice è sbagliato:

```
public static int calcola() {  
    return 1;  
}  
  
public static double calcola() {  
    return 1.0;  
}
```

Per Java le chiamate sarebbero ambigue:

```
calcola();
```

Quale dei due dovrebbe scegliere?

Parametri in ordine diverso

L'ordine conta.

```

public static void mostra(String nome, int eta) {
    System.out.println(nome + ", " + eta);
}

public static void mostra(int eta, String nome) {
    System.out.println(nome + ", " + eta);
}

```

Questi due metodi sono diversi per Java.

Pero non sempre è una buona idea: chi legge potrebbe confondersi. Usa l'overloading solo quando rende il codice più chiaro.

Esempio: calcolare il totale

```

public class Totali {
    public static double totale(double prezzo, int quantita) {
        return prezzo * quantita;
    }

    public static double totale(double prezzo, int quantita, double
spedizione) {
        return prezzo * quantita + spedizione;
    }

    public static void main(String[] args) {
        System.out.println(totale(2.50, 4));
        System.out.println(totale(2.50, 4, 3.90));
    }
}

```

Output:

```

10.0
13.9

```

Il nome `totale` resta lo stesso perché l'idea è la stessa. Cambiano solo i dati disponibili.

Quando usarlo

L'overloading è utile quando:

- i metodi fanno la stessa azione generale
- cambiano solo i parametri disponibili
- il nome comune aiuta a leggere meglio

Evitalo se i metodi fanno cose davvero diverse. In quel caso scegli nomi diversi.